Smart Warehouse WMS: ER Schema and API Specification

Version 2.0 · 28 October 2026 · Replaces v1.0 (the shelf-locking and queue design)

1. Scope and assumptions

- One admin per warehouse. This is a deliberate assumption: it removes concurrent order creation and the locking that would come with it.
- The admin enters outbound orders from the dashboard. Branch self-submission is out of scope.
- Every shelf holds one product at a time. Shelves are grouped by product type.
- All quantities are in cartons.
- Orders run in parallel. There is no shelf locking, no reservation queue and no waiting on other orders.
- Stack: ASP.NET Core Web API in layers (controller, service, repository) with EF Core and SQL Server. The admin dashboard is an ASP.NET MVC app that calls the same API as every other client. Email login with bearer-token auth.
- The admin dashboard refreshes by polling. The ESP32 LED controllers poll the API over HTTP.

2. The availability rule

Stock is tracked per shelf with two counters:

- OnHand: cartons physically on the shelf.
- Committed: cartons promised to unfulfilled order lines.
- Available = OnHand − Committed.

A product's total available stock on the dashboard is the sum of Available across the shelves holding it. A new order is checked against Available, never raw OnHand. Both counters move together at every confirmed pick (section 4.2), so Available stays correct at every moment.

3. ER schema

User

Column	Notes
Id	PK
Email	unique, used to log in
PasswordHash	
FullName	
Role	Admin or Worker
IsActive	

Branch

Id, Name. Destination of outbound orders.

ProductType

Id, Name. Groups shelves and products (cleaning supplies, canned food, and so on).

Product

Column	Notes
Id	PK
Name	
ProductTypeId	FK
PieceBarcode	
CartonBarcode	scanned during inbound

Shelf

Column	Notes
Id	PK
Code	label such as R2-L3
Rack, Level	pick lists are ordered by rack, then level
ProductTypeId	FK, the group this shelf is designated for
ProductId	FK, nullable, what it currently holds
MaxCapacity	cartons
OnHand	cartons, default 0
Committed	cartons, default 0
NodeId	FK to MapNode, the aisle point in front of the shelf
LedDeviceId, LedIndex	which ESP32 and which LED

Derived values: Available = OnHand − Committed. IsFull = OnHand ⩾ MaxCapacity. IsEmpty = OnHand = 0 and Committed = 0. ProductId is set when stock is first placed on an empty shelf and cleared when the shelf becomes empty again, which is what keeps one product per shelf.

Order (outbound)

Column	Notes
Id	PK
BranchId	FK
Deadline	
Status	Assigned, InProgress, AwaitingHandoff, Completed, Cancelled
CreatedByUserId	the admin
CreatedAt, FirstPickAt, CompletedAt, CancelledAt	

OrderLine

Column	Notes
Id	PK
OrderId	FK
ShelfId	FK, chosen by the admin at creation
ProductId	FK
QtyCartons	
Status	Pending or Picked
PickSequence	rack-then-level order within the order
AssignedWorkerId, PickedAt, HandedOffAt	

Line assignment

There is no separate assignment table. Each OrderLine carries AssignedWorkerId, chosen by the admin when the order is created, and every line must have one. The admin can move a pending line to another worker later. There is no automatic load balancing. The

workers on an order are the distinct AssignedWorkerId values of its lines. The picker of a line is its assigned worker.

InboundOrder

Column	Notes
Id	PK
VoucherNumber	receipt voucher
ProductId	FK, found by scanning the barcode
QtyCartons	
TargetShelfId	FK, nullable until assigned
Status	Received, ShelfAssigned, Placed
CreatedByWorkerId, CreatedAt, PlacedAt	the timestamps serve as batch dates for old-versus-new stock

MapNode and MapEdge

- MapNode: Id, X, Y (pixels on the exported map image), Type (Aisle, ShelfAnchor, Dock).
- MapEdge: FromNodeId, ToNodeId, undirected. Weight is the Euclidean distance between the two nodes, computed in code.

4. Core operations

4.1 Create an outbound order (admin)

1. The admin selects branch, deadline, lines (shelf, product, cartons) and the worker for each line. Every line must have a worker.
2. The service validates every line: Available on the chosen shelf must cover the quantity, and the shelf must hold that product.
3. Only if all lines pass, the write runs inside one database transaction. For each line: UPDATE Shelf SET Committed = Committed + @qty WHERE Id = @shelfId AND OnHand − Committed >= @qty.

4. If any statement affects zero rows, the transaction rolls back and the API returns 409 naming the short shelf. Nothing is left committed.
5. On success the order, its lines and the assignments are saved, and the assigned workers are notified.

The check in step 3 is redundant with step 2 by design. It covers any gap between validation and write, and the transaction covers partial failure from any cause (timeout, exception, dropped connection, double submit).

4.2 Confirm a pick (worker)

The line's assigned worker scans the product at the shelf; anyone else gets 403. In one transaction the service sets OnHand = OnHand − qty and Committed = Committed − qty, marks the line Picked, and moves the order to InProgress on its first pick (recording FirstPickAt). If every line is now Picked, the order becomes AwaitingHandoff. The shelf's LED is switched off.

4.3 Cancel an order (admin)

Allowed only while the order is Assigned, meaning before the first confirmed pick. Every line's Committed is released in one transaction and the order becomes Cancelled. After the first pick, cancel is refused with 409.

4.4 Reassign a line (admin)

The admin can move any line that is still Pending to another worker. A line that is already Picked cannot be reassigned.

4.5 Handoff (worker)

After a worker's last pick on an order, the app shows the route to the loading dock. The worker drops their part of the order at the dock and confirms, and their lines get HandedOffAt. The order becomes Completed when every line has been handed off. Stock is not touched, because it was already deducted at pick time.

4.6 Inbound (Android app plus dashboard)

1. The worker opens the inbound screen in the app, scans the incoming product's barcode, and enters cartons and the voucher number. Status: Received. Stock is not counted yet.

2. The admin assigns a shelf from the dashboard: the designated shelf for that product type, or the closest empty shelf if the designated one is full. Closest is measured by path distance on the map graph.
3. The worker places the cartons and confirms. OnHand increases, ProductId is set if the shelf was empty, and the status becomes Placed.

This inbound flow is confirmed.

4.7 Pick-to-light

Each ESP32 polls the API for the LEDs it should light. An LED is lit for each active worker's next Pending line in pick sequence, and is switched off when that pick is confirmed. With parallel orders, several LEDs can be lit at once.

4.8 Routing

Dijkstra over the in-memory graph, loaded at startup. The route covers the calling worker's own remaining lines in PickSequence order, then the dock. The phone sends the shelf the worker is standing next to as the starting anchor and asks again after each pick. The phone scales the map image to its screen and multiplies each point by (screen width ÷ image width) and the same for height.

5. API endpoints

Method	Path	Role	Purpose
POST	/api/auth/login	any	Email login, returns token
GET	/api/products	any	List products
GET	/api/products/by-barcode/{code}	worker	Resolve a scanned barcode
GET	/api/shelves	admin	Shelves with capacity, OnHand, Committed, Available, empty and full flags
GET	/api/inventory	admin	Per-product totals across shelves

Method	Path	Role	Purpose
GET	/api/workers	admin	Workers available for assignment
POST	/api/orders	admin	Create order (4.1)
GET	/api/orders	admin	List with status filter
GET	/api/orders/{id}	admin, assigned worker	Order detail with lines
POST	/api/orders/{id}/cancel	admin	Cancel (4.3)
PUT	/api/orders/{id}/lines/{lineId}/worker	admin	Reassign a pending line to another worker (4.4)
GET	/api/my/orders	worker	Orders containing lines assigned to the caller (own lines only)
POST	/api/orders/{id}/lines/{lineId}/pick	assigned worker	Confirm pick (4.2)
POST	/api/orders/{id}/handoff	assigned worker	Confirm handoff of the caller's lines (4.5)
POST	/api/inbound	worker	Create inbound record
GET	/api/inbound	admin, worker	List inbound records
PUT	/api/inbound/{id}/shelf	admin	Assign target shelf
POST	/api/inbound/{id}/place	worker	Confirm placement
GET	/api/map	worker	Map image URL and size
GET	/api/orders/{id}/route?anchorShelfId={id}	assigned worker	Legs to the remaining shelves, then the dock
GET	/api/devices/{deviceId}/leds	device	LEDs to light

Route response: orderId, then legs. Each leg has sequence, kind (shelf or dock), shelfId and lineId for shelf legs, points (a list of x,y pixel coordinates) and distance.

6. Errors

Code	When
400	Invalid input
403	Worker is not assigned to the line or order
404	Unknown order, line, shelf or barcode
409	Insufficient available stock (body names shelfId, requested, available), cancel after first pick, shelf full on placement, one-product-per-shelf violation
422	Route cannot be computed because a node is unreachable (body names the node)

7. Open items and confirmations

1. Confirmed: handoff is per worker per order (4.5), and the order completes when every line is handed off.
2. Confirmed: the inbound lifecycle in 4.6 (stock is counted at placement, and the admin assigns the shelf).
3. The map image has been exported. Measuring node coordinates, seeding the graph and validating connectivity (design document, Appendix A) happens after the documentation is final and before implementation starts. Every shelf anchor and the dock must be connected in the graph.